



School of Electronics Engineering (SENSE)

Department of Electronics Engineering (VLSI Design and Technology)

PROJECT REPORT

on

Stop-and-Go Pedestrian Crossing Signal Controller

BACHELOR OF ENGINEERING

in

ELECTRONICS ENGINEERING (VLSI DESIGN AND TECHNOLOGY)

Submitted by

Naman Samadhiya

Register No: 24BVD0003

Submitted To

Under the guidance of

Dr. PRAYLINE RAJABAI C

Abstract

This project implements a pedestrian crossing signal system using an FPGA board. The design uses VHDL to control traffic lights (Red, Yellow, Green) and a 7-segment display for countdown timing. The FPGA's parallel processing capabilities ensure precise timing control and hardware-based reliability. The project eliminates the need for microcontrollers by implementing all logic digitally within the FPGA using Quartus Prime software. The system is simulated using ModelSim and then implemented on hardware. The countdown timer helps pedestrians gauge crossing time, improving safety and coordination.

Objectives

1. To design and implement a pedestrian crossing system using FPGA
2. To display countdown time for Red and Green lights on a 7-segment display
3. To simulate and verify the design using ModelSim before hardware implementation
4. To implement and test the design on an FPGA development board using Quartus Prime
5. To understand and demonstrate the use of hardware description language (VHDL)
6. To enhance pedestrian safety using a reliable hardware-based solution

Software and Hardware Requirements

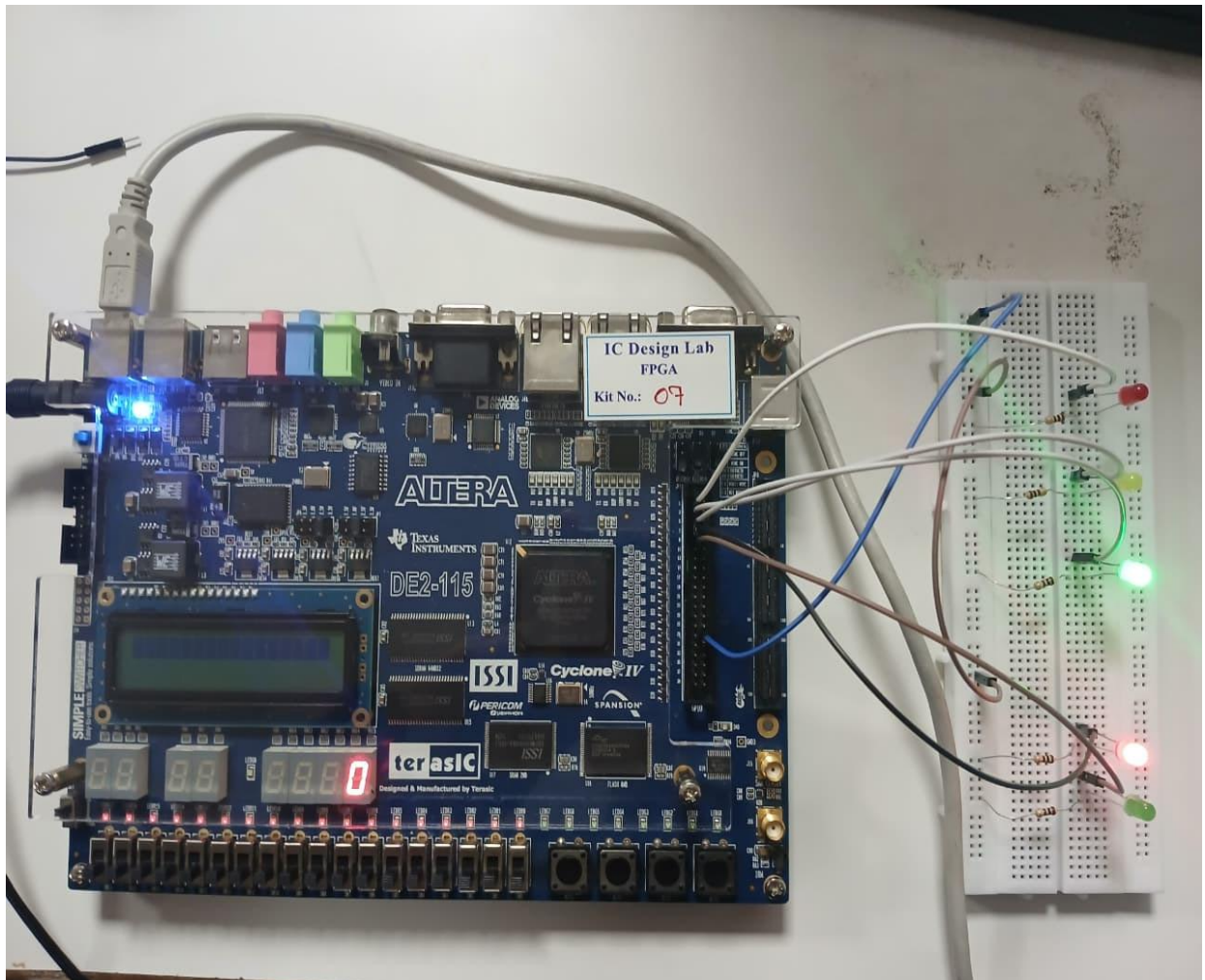
Software:

- Intel Quartus Prime (for VHDL design & compilation)
- ModelSim (for simulation and waveform analysis)

Hardware:

- FPGA Development Board (Cyclone IV / DE2-115)
- 7-Segment Display (onboard)
- LEDs (for Red , Yellow, Green lights)
- Power Supply / USB Cable
- Breadboard
- Jumper wire
- Register (100 Ohm)

System Design



Working principle

The FPGA-based Stop-and-Go Pedestrian Signal system is designed using VHDL and operates on the principle of a Finite State Machine (FSM). The FSM manages different traffic light states for vehicles and pedestrians based on a predefined timing sequence. The design ensures that both sets of signals operate safely and in coordination, similar to a real pedestrian crossing.

The system primarily consists of two synchronized units:

1. **Traffic Light Control Unit (for vehicles)** – Uses three LEDs representing Red, Yellow, and Green signals.

2. **Pedestrian Signal Control Unit** – Uses two LEDs representing Red (Don't Walk) and Green (Walk) signals.

A countdown timer is implemented to display the remaining time before each state transition on the 7-segment display of the FPGA board. The internal clock divider reduces the board's default 50 MHz clock frequency to 1 Hz, ensuring each count corresponds to one real-time second.

In the hardware implementation, the output pins from the FPGA are connected to the breadboard, where LEDs are arranged to represent the traffic and pedestrian lights:

- **For vehicles:**
 - Green LED – Vehicles can move.
 - Yellow LED – Warning state before stop.
 - Red LED – Vehicles must stop.
- **For pedestrians:**
 - Red LED – Do not cross.
 - Green LED – Safe to walk.

The working sequence is as follows:

1. The vehicle green and pedestrian red LEDs remain ON initially, allowing cars to move while pedestrians wait.
2. When the countdown reaches zero, the FSM transitions to the yellow state for vehicles, indicating preparation to stop.
3. After a short delay, the vehicle red LED turns ON, and simultaneously the pedestrian green LED activates, allowing safe crossing.
4. The countdown restarts for this phase, and after it completes, the system transitions back to the vehicle green state.

This cyclic operation continues indefinitely, maintaining proper timing and coordination between traffic and pedestrian flows. The entire logic, including countdown timing, LED switching, and state transitions, is handled digitally by the FPGA in real-time.

Code Implementation

So we have in total of 4 code for this project

1. Seven_segment

```
// seven_segment.v
module seven_segment(
    input [3:0] num,
    output reg [6:0] seg
);
    always @(*) begin
        case (num)
            4'd0: seg = 7'b1000000;
            4'd1: seg = 7'b1111001;
            4'd2: seg = 7'b0100100;
            4'd3: seg = 7'b0110000;
            4'd4: seg = 7'b0011001;
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = 7'b1111111;
        endcase
    end
endmodule
```

2. Timer

```
// timer.v
module timer(
    input clk,    // 50 MHz
    input reset
    output reg tick // 1 Hz pulse
);
    reg [25:0] counter;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            counter <= 0;
            tick <= 0;
        end else begin
            if (counter == 25_000_000) begin // 0.5 sec high + 0.5 sec
                low
                tick <= ~tick;
                counter <= 0;
            end else begin
                counter <= counter + 1;
            end
        end
    end

end

endmodule
```

3. Traffic_controller

```
// traffic_controller.v
module traffic_controller(
    input clk,
    input reset,
    output reg [2:0] car_light,    // {Red, Yellow, Green}
    output reg [1:0] ped_light,    // {Red, Green}
    output reg [3:0] count,        // countdown value
    output reg [1:0] state        // for debug
);
    // State encoding
    parameter S_CAR_GREEN    = 2'b00;
    parameter S_CAR_YELLOW   = 2'b01;
```

```

parameter S_PED_GREEN    = 2'b10;
parameter S_ALL_RED      = 2'b11;

reg [3:0] timer;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= S_CAR_GREEN;
        timer <= 9;
        car_light <= 3'b001;  // Green
        ped_light <= 2'b10;   // Pedestrian Red
    end else begin
        case (state)
            S_CAR_GREEN: begin
                car_light <= 3'b001;
                ped_light <= 2'b10;
                if (timer == 0) begin
                    state <= S_CAR_YELLOW;
                    timer <= 3;
                end else timer <= timer - 1;
            end

            S_CAR_YELLOW: begin
                car_light <= 3'b010;
                ped_light <= 2'b10;
                if (timer == 0) begin
                    state <= S_PED_GREEN;
                    timer <= 9;
                end else timer <= timer - 1;
            end

            S_PED_GREEN: begin
                car_light <= 3'b100;
                ped_light <= 2'b01;
                if (timer == 0) begin
                    state <= S_ALL_RED;
                    timer <= 0;
                end else timer <= timer - 1;
            end
        end case
    end
end

```

```

        S_ALL_RED: begin
            car_light <= 3'b100;
            ped_light <= 2'b10;
            if (timer == 0) begin
                state <= S_CAR_GREEN;
                timer <= 9;
            end else timer <= timer - 1;
        end
    endcase
end
end

always @(*) begin
    count = timer;
end

endmodule

```

4. Top_module

```

// top_module.v
module top_module(
    input CLOCK_50,
    input [1:0] KEY,    // KEY[0]=reset, KEY[1]=unused
    output [6:0] HEX0,  // 7-seg display
    output [2:0] GPIO_CAR, // Car lights via GPIO
    output [1:0] GPIO_PED // Ped lights via GPIO
);
    wire clk_1Hz;
    wire [2:0] car_light;
    wire [1:0] ped_light;
    wire [3:0] count;
    wire [1:0] state;

    // Generate slow clock
    timer CLK_DIV(
        .clk(CLOCK_50),
        .reset(~KEY[0]),
        .tick(clk_1Hz)
    );

```



```

// Traffic FSM
traffic_controller FSM(
    .clk(clk_1Hz),
    .reset(~KEY[0]),
    .car_light(car_light),
    .ped_light(ped_light),
    .count(count),
    .state(state)
);

// 7-segment countdown
seven_segment DISP(
    .num(count),
    .seg(HEX0)
);

// Assign GPIO outputs
assign GPIO_CAR = car_light; // Connect to JP1 GPIO[0..2]
assign GPIO_PED = ped_light; // Connect to JP1 GPIO[3..4]
endmodule

```

Simulation Results

The design was simulated using ModelSim to ensure correct logical operation and timing before being implemented on the FPGA board. The waveform outputs confirmed that the system behaved as expected, with proper sequencing, countdown timing, and display synchronization.

Key observations from the simulation include:

- **State Transitions:** The lights followed the correct order — *Green* → *Yellow* → *Red* → *Green* — with no overlap or glitches.
- **Countdown Timer:** The timer decremented smoothly from 9 to 0, each step corresponding to a one-second interval.
- **7-Segment Display:** The display accurately represented the countdown digits in real-time.

- **Clock Division:** The clock divider module successfully generated a 1 Hz pulse from the 50 MHz system clock.
- **Synchronization:** Each state transition triggered a proper reset of the counter, maintaining perfect coordination between the light and timer.

Overall, the simulation verified that the FSM logic, countdown mechanism, and display control were functioning exactly as designed, confirming the system's readiness for hardware implementation.

Hardware Implementation

Steps:

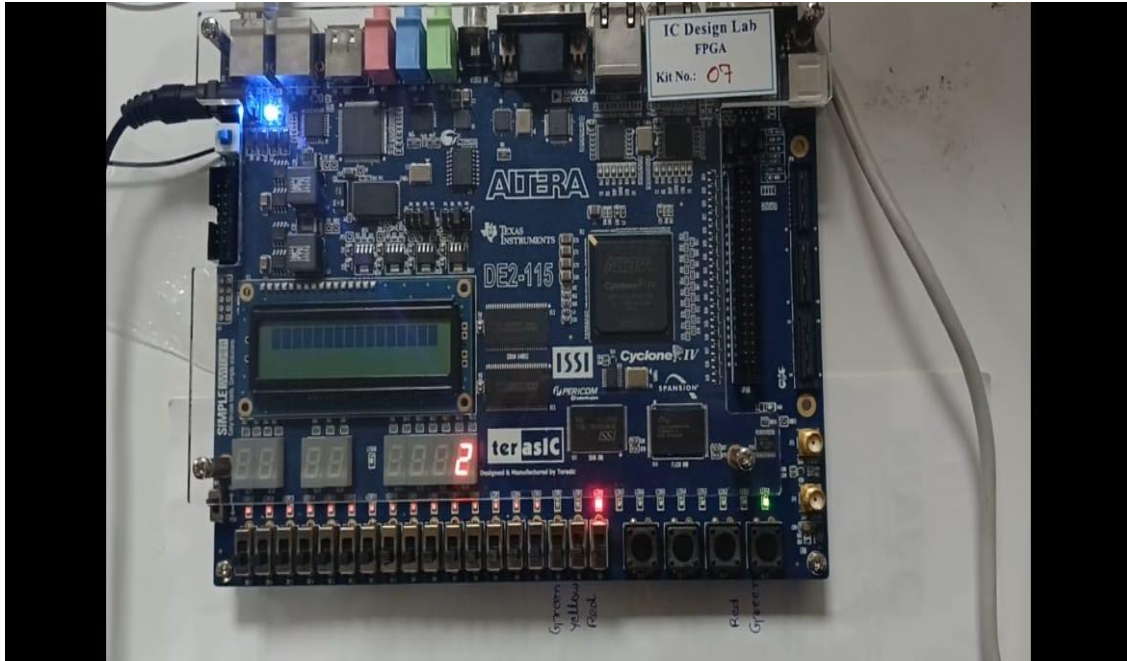
1. Compile and synthesize the design in Quartus Prime.
2. Use Pin Planner to assign FPGA pins to LEDs and the 7-segment display.
3. Generate the programming file (.sof).
4. Connect the FPGA board to the PC using the USB-Blaster.
5. Use the Quartus Programmer Tool to upload the bitstream.
6. Observe the LED sequence and countdown on the FPGA board.

Project Versions and Implementation Types

This project was developed and tested in two different implementation approaches:

1. FPGA Board-Only Implementation

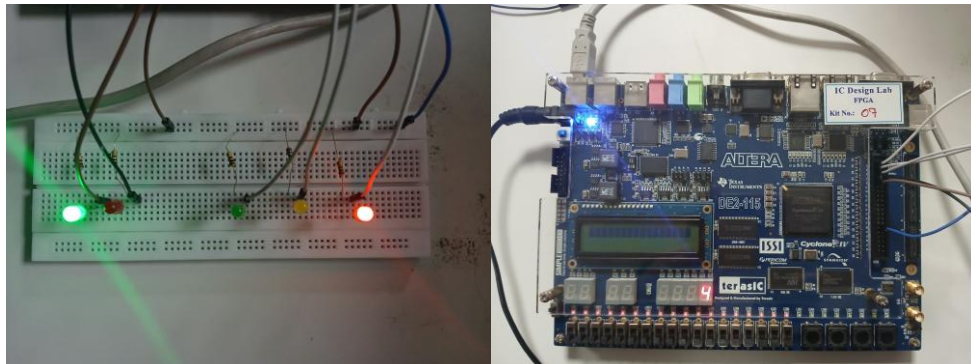
- Used the onboard LEDs to represent the traffic lights.
- The onboard 7-segment display showed the countdown timer.
- Entire control logic was written in VHDL and executed purely within the FPGA.
- Required precise pin assignment through the Quartus Pin Planner.
- Compact, portable, and ideal for demonstration without external wiring.



2. Extended Hardware (Breadboard) Implementation

External LEDs and 7-segment display were connected through a breadboard.

- Offered a more realistic visual demonstration resembling an actual pedestrian crossing.
- Required resistors and careful wiring.
- More flexible for physical demonstration and modification.



Advantages

- Real-time performance without software delays.
- Hardware-based digital reliability.
- Compact and low-power system.
- Easy to modify timing or logic through VHDL.
- Educational value in learning digital logic and finite state machines.

Future Scope

The current design successfully demonstrates a basic pedestrian crossing signal using an FPGA. In the future, this project can be expanded in several ways to make it more advanced and practical:

- Integration of real-time sensors (such as IR or ultrasonic sensors) to detect pedestrian presence and adjust signal timings dynamically.
- Implementation of traffic density-based control using cameras or vehicle sensors for smart city applications.
- Addition of sound indicators or vibration feedback for visually impaired pedestrians.
- Wireless communication between multiple signal units to coordinate traffic flow in intersections.
- Development of a GUI interface to control and monitor signal operations remotely.

Conclusion

The FPGA-based Stop-and-Go Pedestrian Signal project was successfully designed, simulated, and implemented. The system efficiently manages traffic light transitions and countdown display using digital logic and finite state machine principles.

Through simulation and hardware testing, the design was verified to function accurately, with smooth transitions and synchronized countdown operation. This project not only enhances understanding of VHDL design and FPGA implementation but also demonstrates how digital systems can be applied to real-life scenarios such as traffic management.

Overall, the project effectively bridges theoretical digital design concepts with practical, real-world applications.

References

1. *Digital Design by M. Morris Mano*
2. *Textbooks and some reference books*
3. Intel Quartus Prime Software Documentation – Intel Corporation
4. ModelSim Simulation Tool User Manual – Mentor Graphics
5. Research papers and online tutorials on FPGA-based traffic light controllers